LAB 1 · WEEKS 1–2

Cloud Account Security, Identity & Access Management

Identity governance and least privilege — LocalStack IAM & Kubernetes RBAC

Lab Learning Outcomes

At the end of this lab, you will be able to:

1. Stand up a **local cloud lab** using Docker and LocalStack (an AWS-compatible simulator).
2. Apply the **principle of least privilege** by replacing root usage with scoped IAM users, groups and policies.
3. Create and test **fine-grained permissions**, distinguishing what an identity is *allowed* versus *denied* to do.
4. Implement and verify **Role-Based Access Control (RBAC)** in Kubernetes, the real enforcement engine.
5. Audit identities and reason about **MFA, access keys and credential hygiene**.

Course & Assessment Mapping

Item	Mapping
Course Learning Outcome	CLO2 — Construct secure cloud operations that safeguard data integrity
Lecture topics	Weeks 1–2 (Fundamentals, Security Architecture) · Weeks 5 & 7 (Access Control, Identity)
Value / skill clusters	VBE3 (Integrity) · SC8 (Integrated Problem-Solving)
Assessment	Lab report (screenshots + CLI output + short answers) — contributes to the Lab Assignment

Lab Arrangement (2 Sessions over 2 Weeks)

Session	Week	Focus
Session A	Week 1	Environment setup + cloud identity with LocalStack IAM (Tasks 1–4)
Session B	Week 2	Enforced access control with Kubernetes RBAC + audit (Tasks 5–7), then the report

> **Note:** Complete Session A fully before Session B. Keep your commands and screenshots as you go — you will submit them in the report at the end of Session B.

Technical Prerequisites

- A laptop with **at least 8 GB RAM** and administrator rights to install software.
- **Docker Desktop** (Windows/macOS) or Docker Engine (Linux) — free.
- **AWS CLI v2** — free command-line tool (we point it at LocalStack, not real AWS).
- **kind** (Kubernetes-in-Docker) and **kubectl** — free (used in Session B).

- Internet access **only for the first download** of the container images; the lab then runs offline.

> **Security tip:** Nothing in this lab connects to a real cloud provider — LocalStack emulates AWS APIs locally, and kind runs Kubernetes inside Docker on your own laptop.

Session A (Week 1) — Cloud Identity with LocalStack

One-Time Environment Setup

Verify Docker is running, then start LocalStack. Run each command in a terminal:

```
# 1. Confirm Docker is installed and running
docker --version

# 2. Start LocalStack (AWS-compatible cloud) in a container
docker run -d --name localstack -p 4566:4566 localstack/localstack

# 3. Confirm it is healthy (should list running services)
curl http://localhost:4566/_localstack/health
```

Point the AWS CLI at LocalStack by creating a helper alias. On macOS/Linux use `awslocal` via a shell function; on Windows PowerShell use the `--endpoint-url` flag shown throughout.

```
# Configure dummy credentials (LocalStack accepts any value)
aws configure set aws_access_key_id test
aws configure set aws_secret_access_key test
aws configure set region us-east-1

# Test: this talks to LocalStack, NOT real AWS
aws --endpoint-url=http://localhost:4566 sts get-caller-identity
```

> **Note:** Save the output of `sts get-caller-identity` — it is your first deliverable screenshot (it proves which identity you are operating as).

Task 1 — Map the Cloud Identity Landscape

Before creating anything, understand the building blocks of cloud identity. Complete the table in your report by filling the 'Purpose' column in your own words.

Concept	AWS term	Purpose (complete this)
All-powerful owner	Root user	_____
Human/app identity	IAM User	_____
Permission bundle	IAM Policy	_____
Collection of users	IAM Group	_____
Temporary identity	IAM Role	_____

Task 2 — Create a Least-Privilege Admin (Stop Using Root)

The root user is a liability. Create a dedicated admin identity and grant permissions through a **group**, never directly to the user.

```
EP='--endpoint-url=http://localhost:4566'

# 2.1 Create a group and attach an admin policy to the GROUP
aws $EP iam create-group --group-name Admins
aws $EP iam attach-group-policy --group-name Admins \
    --policy-arn arn:aws:iam::aws:policy/AdministratorAccess

# 2.2 Create your personal admin user (replace YOURNAME)
aws $EP iam create-user --user-name CloudAdmin_YOURNAME

# 2.3 Put the user in the group (permissions flow from the group)
aws $EP iam add-user-to-group --group-name Admins \
    --user-name CloudAdmin_YOURNAME

# 2.4 Verify the membership
aws $EP iam get-group --group-name Admins
```

> **Security tip:** Attaching policies to groups (not users) is how you keep permissions manageable and auditable at scale — change the group once, and every member updates.

Task 3 — Enforce Least Privilege with a Scoped Policy

Now create a **read-only** user for a teammate who should never modify data. This demonstrates fine-grained authorization.

```
# 3.1 Create a read-only user
aws $EP iam create-user --user-name Analyst_YOURNAME

# 3.2 Attach a scoped, read-only policy (S3 read only)
aws $EP iam attach-user-policy --user-name Analyst_YOURNAME \
    --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# 3.3 List what the user can do
aws $EP iam list-attached-user-policies --user-name Analyst_YOURNAME
```

In your report, explain: if the Analyst account were stolen, why is the damage limited compared to a stolen admin account? Connect your answer to **blast-radius reduction**.

Task 4 — Credential Hygiene & Access Keys

Programmatic access uses access keys. Create one, then reason about the risk of long-lived keys.

```
# 4.1 Create an access key for the Analyst
aws $EP iam create-access-key --user-name Analyst_YOURNAME

# 4.2 List access keys (note the AccessKeyId and status)
aws $EP iam list-access-keys --user-name Analyst_YOURNAME

# 4.3 Rotate: deactivate the old key (paste the AccessKeyId)
aws $EP iam update-access-key --user-name Analyst_YOURNAME \
    --access-key-id <PASTE_KEY_ID> --status Inactive
```

> **Caution:** In real AWS, never create access keys on the root user and never commit keys to code repositories. Prefer short-lived roles over long-lived keys.

> **Note:** End of Session A. Save your terminal outputs and screenshots. Stop the container if you wish (`docker stop localstack`) and restart it next week with `docker start localstack`.

Session B (Week 2) — Enforced Access Control with Kubernetes RBAC

LocalStack teaches the mechanics of IAM, but it does not fully *enforce* policies. Kubernetes RBAC does — so in this session you will see access control actually block an unauthorised action.

Setup — Create a Local Kubernetes Cluster

```
# Create a throwaway cluster (runs inside Docker)
kind create cluster --name ccse-lab1

# Confirm it is up
kubectl cluster-info --context kind-ccse-lab1
kubectl get nodes
```

Task 5 — Separate Environments with Namespaces

```
kubectl create namespace dev
kubectl create namespace prod
kubectl get namespaces
```

Task 6 — Define a Role and Bind It (Least Privilege)

Create a role that can only **read** pods in `dev`, and bind it to a test service account. This is RBAC: a role (permissions) plus a role-binding (who gets them).

```
# 6.1 Create a service account to represent a developer
kubectl create serviceaccount dev-user -n dev

# 6.2 Create a Role that allows only get/list/watch on pods in dev
kubectl create role pod-reader -n dev \
    --verb=get,list,watch --resource=pods

# 6.3 Bind the Role to the service account
kubectl create rolebinding dev-user-binding -n dev \
    --role=pod-reader --serviceaccount=dev:dev-user
```

Task 7 — Test That Access Control Works

Use `kubectl auth can-i` to prove the boundary. Record every result.

```
SA=system:serviceaccount:dev:dev-user

# Should be YES — reading pods in dev is allowed
kubectl auth can-i list pods -n dev --as=$SA

# Should be NO — deleting pods is not granted
kubectl auth can-i delete pods -n dev --as=$SA
```

```
# Should be NO — the role does not extend to prod
kubectl auth can-i list pods -n prod --as=$SA
```

> **Security tip:** This is least privilege enforced by the platform: the developer can do exactly what the role permits and nothing more — even in the same cluster, `prod` is off-limits.

In your report, relate the three `can-i` results to authentication versus authorization: which step is the service account passing, and which step is blocking the delete and the prod access?

Deliverables & Assessment

1. Screenshots (label each clearly)

- Output of `sts get-caller-identity` showing your operating identity.
- `get-group Admins` output showing your CloudAdmin user as a member.
- `list-attached-user-policies` for the Analyst showing only the read-only policy.
- The three `kubectl auth can-i` results (YES / NO / NO).

2. Short-Answer Questions

Q1. Why is attaching policies to **groups** better than attaching them directly to users?
Q2. What is the difference between an IAM **User** and an IAM **Role**?
Q3. Explain **least privilege** using the Analyst account, and how it reduces blast radius if compromised.
Q4. In Kubernetes, what is the difference between a **Role** and a **RoleBinding**?
Q5. Why did the developer service account fail to access `prod`, and which security principle does that demonstrate?

3. Verification Command

Paste the output of the following to prove your cluster RBAC is in place:

```
kubectl get rolebinding dev-user-binding -n dev -o yaml
```

Security Best-Practices Checklist

- ☐ Root user is not used for daily tasks (a dedicated admin identity exists).
- ☐ Permissions are granted via groups/roles, not directly to individual users.
- ☐ At least one least-privilege (read-only) identity was created and tested.
- ☐ Access keys were listed and a rotation (deactivate) was demonstrated.
- ☐ Kubernetes RBAC blocks an unauthorised action (delete / cross-namespace).

Cleanup & Teardown

```
# Remove the Kubernetes cluster
kind delete cluster --name ccse-lab1

# Stop and remove LocalStack
docker stop localstack && docker rm localstack
```

Expansion Ideas (Advanced Students)

- **Infrastructure as Code:** recreate the IAM group, user and policy attachment using a Terraform script pointed at LocalStack.

- **Policy conditions:** write a custom IAM policy JSON that denies all actions unless a condition (e.g. `aws:MultiFactorAuthPresent`) is met, and attach it.
- **RBAC depth:** create a `ClusterRole` + `ClusterRoleBinding` and compare its scope to a namespaced `Role`.
- **Policy-as-code guardrails:** install OPA Gatekeeper and write a policy that blocks any pod running as root.

References

- Course lectures — Week 1 (Fundamentals), Week 2 (Security Architecture), Week 5 (Access Control), Week 7 (Identity Management).
- LocalStack documentation — docs.localstack.cloud
- Kubernetes RBAC — kubernetes.io/docs/reference/access-authn-authz/rbac
- CSA Security Guidance v5 — Domain on Identity & Access Management.